

Working the Algorithms by Hand

Pen-and-paper exercises for eight computational-biology architectures

A companion to the eight-notebook tour. Every number below was designed to be small enough to compute with a pencil (and, where logs or exponentials appear, a basic calculator).

Each architecture rests on a computation whose *core* is genuinely hand-sized: a table you fill in, a single training step, or one key statistic. What is *not* hand-sized is large-scale iterative optimisation and big linear algebra. The honest split:

Architecture	What you <i>can</i> do by hand (here)	What you cannot (and why)
Dynamic programming	Fill the full alignment matrix + traceback	— (nothing; it is exact and small)
Hidden Markov model	Viterbi + forward on a length-3 sequence	Baum–Welch training (many EM iterations)
SVM / kernels	Kernel values, the feature-map identity, a kernel matrix	Solving the margin QP for many points
Tree ensembles	One Gini split; one gradient-boosting step	Growing a full forest / many boosting rounds
CNN	One convolution + ReLU + pool (forward)	Learning the filter weights (back-prop)
Transformer	One self-attention head (forward)	Training Q, K, V projections
Graph neural net	One message-passing layer (forward)	Training over many layers/graphs
AlphaFold pieces	Mutual information; RMSD	DCA matrix inversion, MDS, Kabsch SVD

The pattern: the **forward computation, a single step, or the central statistic** is hand-tractable; **iterative learning and large matrix factorisations** are not. Answers to the “Try it yourself” problems are on the last page.

1. Dynamic programming — Needleman–Wunsch global alignment

Worldview: biology is a string; similarity is the cheapest way to edit one sequence into another. The algorithm fills a grid where each cell holds the best score of aligning two prefixes.

Data. Sequences $X = \text{GACT}$ (top) and $Y = \text{GAT}$ (side). Scoring: match = +1, mismatch = -1, gap = -1.

Exercise. Fill the 4×5 score matrix H , then trace back to read off the optimal alignment and its score.

Worked solution

Recurrence. With $H_{0,0} = 0$, first row/column are pure gaps ($H_{0,j} = -j$, $H_{i,0} = -i$), and

$$H_{i,j} = \max \begin{cases} H_{i-1,j-1} + s(Y_i, X_j) & \text{(diagonal: align)} \\ H_{i-1,j} - 1 & \text{(up: gap in } X) \\ H_{i,j-1} - 1 & \text{(left: gap in } Y) \end{cases} \quad s = \begin{cases} +1 & \text{match} \\ -1 & \text{mismatch} \end{cases}$$

Example cell. $H_{2,2}$ compares $Y_2 = \text{A}$ with $X_2 = \text{A}$ (match, +1):

$$H_{2,2} = \max\{H_{1,1} + 1, H_{1,2} - 1, H_{2,1} - 1\} = \max\{1 + 1, 0 - 1, 0 - 1\} = 2.$$

Completed matrix (shaded cells are the traceback path):

	--	G	A	C	T
--	0	-1	-2	-3	-4
G	-1	1	0	-1	-2
A	-2	0	2	1	0
T	-3	-1	1	1	2

Traceback (from bottom-right $H_{3,4} = 2$, following the move that produced each value):

- $H_{3,4} = 2$: T/T match, came diagonally from $H_{2,3} = 1$.
- $H_{2,3} = 1$: came from the *left* ($H_{2,2} = 2$, minus a gap) $\Rightarrow$ the C in X aligns to a gap.
- $H_{2,2} = 2$: A/A match, diagonal from $H_{1,1} = 1$.
- $H_{1,1} = 1$: G/G match, diagonal from $H_{0,0} = 0$.

X: G A C T Y: G A - T	score = +1 + 1 - 1 + 1 = 2
--------------------------	-----------------------------------

Exercise. *Try it yourself (1).* Re-score the same alignment with gap = -2 (match/mismatch unchanged). Does G A C T / G A -- T still win, or does a gapless alternative tie/beat it?

2. Hidden Markov model — Viterbi and forward

Worldview: an observed sequence is produced by a hidden chain of states that switches stochastically; each state emits symbols with its own bias.

Data. Two states C (coding) and N (noncoding); two emitted symbols $\{\text{G}, \text{A}\}$. Initial $\pi = (0.5, 0.5)$.

$$A = \begin{array}{c|cc} & C & N \\ \hline C & 0.7 & 0.3 \\ N & 0.4 & 0.6 \end{array} \quad B = \begin{array}{c|cc} & G & A \\ \hline C & 0.6 & 0.4 \\ N & 0.2 & 0.8 \end{array}$$

Observed sequence $O = (\text{G}, \text{A}, \text{G})$.

Exercise. (a) Use the **Viterbi** recursion to find the most probable hidden-state path and its probability. (b) Use the **forward** recursion to find $P(O)$.

Worked solution — (a) Viterbi

$\delta_t(s)$ = probability of the best path ending in state s at time t . Initialise with $\delta_1(s) = \pi_s B_s(O_1)$:

$$\delta_1(C) = 0.5 \times 0.6 = 0.30, \quad \delta_1(N) = 0.5 \times 0.2 = 0.10.$$

Recurse $\delta_t(s) = B_s(O_t) \cdot \max_k \{\delta_{t-1}(k) A_{k,s}\}$. At $t = 2$ (emit **A**):

$$\delta_2(C) = 0.4 \cdot \max\{0.30(0.7), 0.10(0.4)\} = 0.4(0.21) = \mathbf{0.084} \quad (\text{from } C),$$

$$\delta_2(N) = 0.8 \cdot \max\{0.30(0.3), 0.10(0.6)\} = 0.8(0.09) = \mathbf{0.072} \quad (\text{from } C).$$

At $t = 3$ (emit **G**):

$$\delta_3(C) = 0.6 \cdot \max\{0.084(0.7), 0.072(0.4)\} = 0.6(0.0588) = \mathbf{0.03528} \quad (\text{from } C),$$

$$\delta_3(N) = 0.2 \cdot \max\{0.084(0.3), 0.072(0.6)\} = 0.2(0.0432) = 0.00864 \quad (\text{from } N).$$

The larger terminal value is $\delta_3(C) = 0.03528$. Back-pointers were $C \leftarrow C \leftarrow C$, so

best path = $C \rightarrow C \rightarrow C$, $P^* = 0.03528$

Worked solution — (b) forward

$\alpha_t(s)$ replaces “max” with “ $\sum$ ”. α_1 equals δ_1 : (0.30, 0.10). Then

$$\alpha_2(C) = 0.4(0.30 \cdot 0.7 + 0.10 \cdot 0.4) = 0.4(0.25) = 0.100, \quad \alpha_2(N) = 0.8(0.30 \cdot 0.3 + 0.10 \cdot 0.6) = 0.8(0.15) = 0.120.$$

$$\alpha_3(C) = 0.6(0.100 \cdot 0.7 + 0.120 \cdot 0.4) = 0.6(0.118) = 0.0708, \quad \alpha_3(N) = 0.2(0.100 \cdot 0.3 + 0.120 \cdot 0.6) = 0.2(0.102) = 0.0204$$

$P(O) = \alpha_3(C) + \alpha_3(N) = 0.0708 + 0.0204 = 0.0912$

Note $P^* = 0.03528 < P(O) = 0.0912$: the single best path carries about 39% of the total probability.

Exercise. Try it yourself (2). Recompute $\delta_2(N)$ if the transition $A_{N,N}$ were 0.9 (and $A_{N,C} = 0.1$). Does the most likely state at $t = 2$ change?

3. SVM and kernels — the kernel trick by hand

Worldview: data that is tangled in its raw coordinates may be separable after mapping into a richer feature space; a *kernel* computes inner products in that space without ever building it.

Worked solution — (a) a kernel equals a dot product of feature maps

Data. $x = (1, 2)$, $z = (3, 1)$. Quadratic kernel $K(x, z) = (x \cdot z)^2$.

Directly,

$$x \cdot z = 1 \cdot 3 + 2 \cdot 1 = 5, \quad K(x, z) = 5^2 = 25.$$

Claim: $K(x, z) = \varphi(x) \cdot \varphi(z)$ with $\varphi(u) = (u_1^2, \sqrt{2}u_1u_2, u_2^2)$. Compute the maps:

$$\varphi(x) = (1, 2\sqrt{2}, 4), \quad \varphi(z) = (9, 3\sqrt{2}, 1),$$

$$\varphi(x) \cdot \varphi(z) = 1 \cdot 9 + (2\sqrt{2})(3\sqrt{2}) + 4 \cdot 1 = 9 + 6 \cdot 2 + 4 = 25 = K(x, z). \quad \checkmark$$

The 3-D dot product and the 2-D kernel agree — but the kernel never formed the 3-D vectors.

Worked solution — (b) XOR becomes separable in one new feature

Data. XOR: points and labels $(0,0):0$, $(1,1):0$, $(1,0):1$, $(0,1):1$.

No straight line separates these in the plane. Add the single feature $u = (x_1 - x_2)^2$:

$$(0,0) \rightarrow 0, \quad (1,1) \rightarrow 0, \quad (1,0) \rightarrow 1, \quad (0,1) \rightarrow 1.$$

Now the rule “ $u > 0.5 \Rightarrow$ class 1” separates them perfectly on a line. A nonlinear boundary in 2-D became a threshold in 1-D — the essence of what a kernel buys you.

Worked solution — (c) an RBF kernel matrix

Data. Points $a = (0,0)$, $b = (1,0)$, $c = (0,2)$; RBF kernel $K(u,v) = e^{-\gamma\|u-v\|^2}$, $\gamma = 0.5$.

Squared distances: $\|a-b\|^2 = 1$, $\|a-c\|^2 = 4$, $\|b-c\|^2 = 1+4 = 5$. Hence

$$K = \begin{pmatrix} 1 & e^{-0.5} & e^{-2} \\ e^{-0.5} & 1 & e^{-2.5} \\ e^{-2} & e^{-2.5} & 1 \end{pmatrix} \approx \begin{pmatrix} 1 & 0.607 & 0.135 \\ 0.607 & 1 & 0.082 \\ 0.135 & 0.082 & 1 \end{pmatrix}.$$

Every diagonal entry is 1 (a point is identical to itself) and similarity decays with distance; a and c (farthest apart) are the least similar.

Exercise. *Try it yourself (3).* With the quadratic map φ of part (a), verify $K(x,x) = (x \cdot x)^2$ for $x = (1,2)$ both directly and via $\varphi(x) \cdot \varphi(x)$.

4. Tree ensembles — a Gini split and one boosting step

Worldview: biology is tabular; signal lives in thresholds and feature interactions. A tree asks yes/no questions; boosting stacks weak trees onto the previous errors.

Worked solution — (a) choosing a split by Gini impurity

Data. One feature x , binary label y : $(x,y) = (1,0), (2,0), (3,1), (4,0), (5,1), (6,1)$.

Gini impurity of a node with class-1 fraction p is $G = 1 - p^2 - (1-p)^2$. The root has 3 zeros and 3 ones, $p = \frac{1}{2}$:

$$G_{\text{root}} = 1 - \frac{1}{4} - \frac{1}{4} = 0.5.$$

For a threshold split, the quality is the *weighted* child impurity; the *gain* is G_{root} minus that. Test three thresholds:

split	left (labels, G)	right (labels, G)	weighted G / gain
$x < 2.5$	$\{0,0\}, G = 0$	$\{1,0,1,1\}, G = 1 - (\frac{3}{4})^2 - (\frac{1}{4})^2 = 0.375$	$\frac{2}{6}(0) + \frac{4}{6}(0.375) = 0.25$ / 0.25
$x < 3.5$	$\{0,0,1\}, G = \frac{4}{9} \approx 0.444$	$\{0,1,1\}, G \approx 0.444$	0.444 / 0.056
$x < 4.5$	$\{0,0,1,0\}, G = 0.375$	$\{1,1\}, G = 0$	0.25 / 0.25

The splits $x < 2.5$ and $x < 4.5$ tie for the best gain (0.25); $x < 3.5$ is much worse because both sides stay mixed. A greedy tree would take either of the two winners as its first question.

Worked solution — (b) one round of gradient boosting (squared loss)

Data. One feature x , continuous target y : $(1, 2), (2, 3), (3, 7), (4, 8)$.

Step 0 — constant model. For squared loss the best constant is the mean:

$$F_0 = \frac{2+3+7+8}{4} = 5.$$

Residuals (negative gradient of $\frac{1}{2}(y - F)^2$ is just $y - F$):

$$r = (2 - 5, 3 - 5, 7 - 5, 8 - 5) = (-3, -2, 2, 3), \quad \text{SSE} = 9 + 4 + 4 + 9 = 26.$$

Fit a stump to the residuals. Try split $x < 2.5$: left mean $= \frac{-3-2}{2} = -2.5$, right mean $= \frac{2+3}{2} = 2.5$:

$$\text{SSE} = (-3 + 2.5)^2 + (-2 + 2.5)^2 + (2 - 2.5)^2 + (3 - 2.5)^2 = 0.25 \times 4 = 1.0,$$

far below the alternatives ($x < 1.5$ and $x < 3.5$ both give $\text{SSE} = 14$). So the stump predicts -2.5 for $x < 2.5$ and $+2.5$ otherwise. **Update** with learning rate $\eta = 0.1$, $F_1 = F_0 + \eta \cdot (\text{stump})$:

$$F_1 = (5 - 0.25, 5 - 0.25, 5 + 0.25, 5 + 0.25) = (4.75, 4.75, 5.25, 5.25).$$

New residuals: $(-2.75, -1.75, 1.75, 2.75)$, $\text{SSE} = 21.25 < 26$. One weak tree nudged every prediction toward its target; repeating this is gradient boosting.

Exercise. *Try it yourself (4).* In part (a), what is the Gini gain of the split $x < 5.5$? Is it as good as the two winners?

5. Convolutional network — one filter, forward pass

Worldview: the same local motif can appear anywhere, so slide one shared detector across the signal (translation invariance).

Data. Input signal $s = [1, 0, 2, 1, 0, 3, 1]$; one filter $w = [1, 0, -1]$ (a simple edge detector), bias 0; “valid” positions only.

Exercise. Compute the convolution (cross-correlation) feature map, apply ReLU, then global max-pooling. Which position responds most strongly?

Worked solution

Slide w over every length-3 window and take the dot product:

$$\begin{aligned} [1, 0, 2] \cdot w &= 1 - 2 = -1, & [0, 2, 1] \cdot w &= 0 - 1 = -1, & [2, 1, 0] \cdot w &= 2 - 0 = 2, \\ [1, 0, 3] \cdot w &= 1 - 3 = -2, & [0, 3, 1] \cdot w &= 0 - 1 = -1. \end{aligned}$$

Feature map $= [-1, -1, 2, -2, -1]$. Apply $\text{ReLU}(t) = \max(0, t)$:

$$[0, 0, \mathbf{2}, 0, 0] \xrightarrow{\text{max-pool}} [2].$$

The filter measures “left minus right.” Its one positive response, at window $[2, 1, 0]$ (a falling edge), is the strongest local pattern; pooling reports that single number regardless of *where* it occurred — the same detector would have fired at any position.

Exercise. *Try it yourself (5).* Replace the filter with $w = [1, 1, 1]$ (a smoother). Compute the new feature map and the pooled value.

6. Transformer — one self-attention head, forward pass

Worldview: every position should be able to look directly at every other and pull in what is relevant, with weights computed from content.

Data. Three tokens with 2-D embeddings $x_1 = (1, 0)$, $x_2 = (0, 1)$, $x_3 = (1, 1)$. To keep the arithmetic by hand we set the projections to the identity, so $Q = K = V = X$; the scaling is $1/\sqrt{d} = 1/\sqrt{2}$.

Exercise. Compute the attention output for query token 1: scores, softmax weights, then the weighted sum of values.

Worked solution

Scores = $\frac{q_1 \cdot k_j}{\sqrt{2}}$ for each key j :

$$\frac{x_1 \cdot x_1}{\sqrt{2}} = \frac{1}{\sqrt{2}} = 0.707, \quad \frac{x_1 \cdot x_2}{\sqrt{2}} = 0, \quad \frac{x_1 \cdot x_3}{\sqrt{2}} = \frac{1}{\sqrt{2}} = 0.707.$$

Softmax of $(0.707, 0, 0.707)$. Exponentiate: $e^{0.707} = 2.028$, $e^0 = 1$, $e^{0.707} = 2.028$; sum = 5.056:

$$\text{weights} = \left(\frac{2.028}{5.056}, \frac{1}{5.056}, \frac{2.028}{5.056} \right) = (0.401, 0.198, 0.401).$$

Output = $\sum_j \text{weight}_j v_j$ with $v_j = x_j$:

$$0.401(1, 0) + 0.198(0, 1) + 0.401(1, 1) = (0.401+0.401, 0.198+0.401) = \boxed{(0.802, 0.599)}.$$

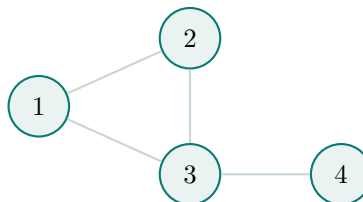
Token 1 attends mostly to itself and to token 3 (both share its first coordinate) and largely ignores the orthogonal token 2 — attention is content-based routing.

Exercise. *Try it yourself (6).* Compute the softmax weights for query token 3, whose raw scores are $(0.707, 0.707, 1.414)$. Which token does it favour?

7. Graph neural network — one message-passing layer

Worldview: biology is relational; a node's meaning comes from its neighbours. A layer averages each node with its neighbours, then transforms.

Data. Four nodes; edges $\{1-2, 1-3, 2-3, 3-4\}$; scalar features $h = (h_1, h_2, h_3, h_4) = (1, 2, 3, 4)$. Add a self-loop to every node, then average over the closed neighbourhood (row-normalised propagation), apply weight $W = 2$ and ReLU.



Exercise. Compute the updated feature of every node after one layer.

Worked solution

Closed neighbourhoods (node + its neighbours): $\tilde{N}(1) = \{1, 2, 3\}$, $\tilde{N}(2) = \{1, 2, 3\}$, $\tilde{N}(3) = \{1, 2, 3, 4\}$, $\tilde{N}(4) = \{3, 4\}$. Average the features:

$$h'_1 = \frac{1+2+3}{3} = 2, \quad h'_2 = \frac{1+2+3}{3} = 2, \quad h'_3 = \frac{1+2+3+4}{4} = 2.5, \quad h'_4 = \frac{3+4}{2} = 3.5.$$

Apply $W = 2$ then ReLU (all positive here):

$$h'' = (4, 4, 5, 7).$$

Notice the four original values $\{1, 2, 3, 4\}$ have been pulled toward local averages — neighbouring nodes now look more alike. Stack many such layers and *every* node converges to the same vector: this is the over-smoothing the deeper experiments showed. (The symmetric normalisation $\hat{A} = \tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2}$ replaces the plain average with $\tilde{A}_{ij} / \sqrt{\tilde{d}_i \tilde{d}_j}$; same idea, messier square roots.)

Exercise. Try it yourself (7). Add an edge 2–4. Recompute h'_4 (its closed neighbourhood becomes $\{2, 3, 4\}$).

8. AlphaFold pieces — mutual information and RMSD

Worldview: contacting residues co-evolve, so correlations between alignment columns reveal 3-D contacts; structures are then scored by how far predicted atoms sit from the truth.

Worked solution — (a) mutual information detects co-evolution

Data. A 4-sequence alignment; compare two columns. *Co-evolving* pair has columns (A, A), (A, A), (T, T), (T, T); an *independent* pair has (A, A), (A, T), (T, A), (T, T).

$$\text{Mutual information } I = \sum_{a,b} p(a, b) \log_2 \frac{p(a, b)}{p(a) p(b)}.$$

Co-evolving pair. Marginals $p(A) = p(T) = \frac{1}{2}$ in each column. Joint: $p(A, A) = p(T, T) = \frac{1}{2}$, the off-diagonal 0. Only the two non-zero cells contribute:

$$I = \frac{1}{2} \log_2 \frac{1/2}{(1/2)(1/2)} + \frac{1}{2} \log_2 \frac{1/2}{(1/2)(1/2)} = \frac{1}{2} \log_2 2 + \frac{1}{2} \log_2 2 = \boxed{1 \text{ bit}}.$$

Independent pair. Every joint cell is $\frac{1}{4} = p(a)p(b)$, so each log is $\log_2 1 = 0$:

$$I = 4 \times \frac{1}{4} \log_2 \frac{1/4}{(1/2)(1/2)} = \log_2 1 = \boxed{0 \text{ bits}}.$$

A column pair that varies *together* carries a full bit; a pair that varies *independently* carries none. Ranking residue pairs by this quantity is the raw contact signal (the notebooks then strip indirect correlations with direct-coupling analysis).

Worked solution — (b) RMSD between two structures

Data. Three atoms. Predicted $P = \{(0, 0, 0), (1, 0, 0), (2, 0, 0)\}$; true $Q = \{(0.5, 0, 0), (0.5, 0, 0), (2.5, 0, 0)\}$ (already superposed).

Root-mean-square deviation = $\sqrt{\frac{1}{N} \sum_i \|P_i - Q_i\|^2}$. Per-atom squared deviations:

$$(0.5)^2 = 0.25, \quad (1 - 0.5)^2 = 0.25, \quad (2 - 2.5)^2 = 0.25.$$

$$\text{RMSD} = \sqrt{\frac{0.25+0.25+0.25}{3}} = \sqrt{0.25} = \boxed{0.5} \text{ (length units).}$$

(The hard part skipped here is finding the optimal rotation that *minimises* this — the Kabsch algorithm — which needs a singular-value decomposition, not a pencil.)

Exercise. *Try it yourself (8).* A third column pair reads (A, A), (A, A), (A, T), (T, T). Its mutual information is between 0 and 1 bit — estimate whether it is closer to a contact or to independence by eye, then (optional) compute it.

Answer key. **(1)** With gap = -2: GACT/GA-T scores 1+1-2+1 = 1; the gapless alignment GACT/GA-T isn't possible (lengths differ), but aligning GAT under GAC (drop T) etc. also tops out at 1, so the gapped alignment still wins or ties — larger gap penalties discourage indels. **(2)** $\delta_2(N) = 0.8 \cdot \max\{0.30(0.3), 0.10(0.9)\} = 0.8(0.09) = 0.072$ (the $C \rightarrow N$ path 0.09 still beats $N \rightarrow N$'s 0.09 tie; here equal, so 0.072 unchanged) — C remains the more likely state at $t=2$ since $\delta_2(C) = 0.084$. **(3)** $x \cdot x = 5 \Rightarrow K = 25$; $\varphi(x) = (1, 2\sqrt{2}, 4)$, $\varphi(x) \cdot \varphi(x) = 1+8+16 = 25$. **(4)** $x < 5.5$: left $\{0, 0, 1, 0, 1\}$ ($G = 1 - (\frac{3}{5})^2 - (\frac{2}{5})^2 = 0.48$), right $\{1\}$ ($G = 0$); weighted = $\frac{5}{6}(0.48) = 0.40$, gain = 0.10 — worse than the 0.25 winners. **(5)** $w = [1, 1, 1]$: feature map $[3, 3, 3, 4, 4]$, pooled = 4. **(6)** $e^{0.707}, e^{0.707}, e^{1.414} = 2.028, 2.028, 4.113$; sum 8.169; weights (0.248, 0.248, 0.503) — token 3 favours itself. **(7)** $h'_4 = \frac{2+3+4}{3} = 3$ (then $W=2$, ReLU $\Rightarrow 6$). **(8)** Joint: $p(\text{AA})=\frac{1}{2}, p(\text{AT})=\frac{1}{4}, p(\text{TT})=\frac{1}{4}$; marginals $p_i(\text{A})=\frac{3}{4}, p_i(\text{T})=\frac{1}{4}$, $p_j(\text{A})=\frac{1}{2}, p_j(\text{T})=\frac{1}{2}$. $I \approx 0.5 \log_2 \frac{0.5}{0.375} + 0.25 \log_2 \frac{0.25}{0.375} + 0.25 \log_2 \frac{0.25}{0.125} \approx 0.31$ bits — partial coupling, nearer independence.